

```
#!/usr/bin/env python3
```

```
"""
```

```
convert_raw_to_markdown.py
```

```
-----
```

PURPOSE FOR BEGINNER DEVELOPERS:

This script monitors an incoming folder of messy notes, downloads, or web scrapes and converts them into clean, standardized Markdown files.

It strips away HTML tags and web interface junk so local AI models only read pure text.

```
"""
```

```
import sys
```

```
from pathlib import Path
```

```
from datetime import datetime, timezone
```

```
import markdownify
```

```
from docx import Document
```

```
RAW_DIR = Path("~/raw-bucket").expanduser()
```

```
OUTPUT_DIR = Path("~/novae-xorpus/01_raw_sources/text").expanduser()
```

```
def convert_docx(path: Path) -> str:
```

```
    doc = Document(str(path))
```

```
    lines = []
```

```
    for p in doc.paragraphs:
```

```
        if p.style.name.startswith("Heading"):
```

```
            try:
```

```
                lvl = int(p.style.name.replace("Heading ", ""))
```

```
                lines.append(f'{"#" * lvl} {p.text}')
```

```
            except ValueError:
```

```
                lines.append(p.text)
```

```
        elif p.text.strip():
```

```
            lines.append(p.text)
```

```
    return "\n\n".join(lines)
```

```
def convert_html(path: Path) -> str:
```

```
    raw = path.read_text(encoding="utf-8", errors="ignore")
```

```
    return markdownify.markdownify(raw, heading_style="ATX")
```

```
def convert_txt(path: Path) -> str:
```

```
    raw = path.read_text(encoding="utf-8", errors="ignore")
```

```
    return f'#{path.stem}\n\n{raw}'
```

```
CONVERTERS = {
```

```
    ".docx": convert_docx,
```

```

".html": convert_html,
".htm": convert_html,
".txt": convert_txt,
".md": lambda p: p.read_text(encoding="utf-8", errors="ignore")
}

def main():
    if not RAW_DIR.exists():
        print(f"[!] Raw directory {RAW_DIR} does not exist.")
        return

    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
    count = 0

    for file in RAW_DIR.iterdir():
        ext = file.suffix.lower()
        if ext in CONVERTERS:
            out_file = OUTPUT_DIR / f"{file.stem}.md"
            print(f"[*] Converting: {file.name} -> {out_file.name}")
            try:
                body = CONVERTERS[ext](file)
                frontmatter = f"---
source_file: {file.name}
ingested_at: {datetime.now(timezone.utc).isoformat()}
tier: 1
---
"

                out_file.write_text(frontmatter + body, encoding="utf-8")
                count += 1
            except Exception as e:
                print(f"[!] Error converting {file.name}: {e}")

    print(f"[✓] Converted {count} files to clean Markdown in {OUTPUT_DIR}")

if __name__ == "__main__":
    main()

```